

Day 7 — Live coding: Banca Lumera da (quasi) zero

Oggi diamo il via all'assistente che questo corso continua a costruire: **Banca Lumera**, un assistente per la clientela di una banca retail. Costruiamo tre capacità conversazionali complete — consultazione del saldo, prenotazione di un appuntamento in filiale, informazioni sui costi delle carte — e non scriviamo **nemmeno una riga di Python**. È questo il punto della giornata: prima ancora che entri in gioco il codice custom, Rasa genera già una quantità sorprendente di comportamento attorno a semplice YAML — le domande, la compilazione degli slot, le risposte, le riparazioni. Oggi si tratta di vedere quella macchinaria in funzione e di sapere esattamente in quale file vive ciascun pezzo.

Lo stato finale di questo tutorial è la cartella `lumera-assistant/` accanto a questo file: se ti perdi, o se vuoi saltare avanti, quella cartella è il risultato funzionante e si addestra ed esegue così com'è.

REQUISITI - `uv` installato (su macOS: `brew install uv`) - una `RASA_LICENSE` valida esportata - una `OPENAI_API_KEY` esportata (il template di oggi salta l'unica chiamata LLM in fase di training, ma ogni conversazione ne ha bisogno)

1. Scaffold dal template tutorial

Nel Day 6 abbiamo generato lo scaffold del template **default** per dare un'occhiata in giro. Oggi vogliamo l'opposto: il minor materiale possibile, così che tutto ciò che è nel progetto finisca per essere nostro. Rasa fornisce diversi template (`default`, `tutorial`, `basic`, `finance`, `telco`); il template **tutorial** è quello quasi vuoto.

Si parte da zero:

```
mkdir lumera-assistant && cd lumera-assistant
uv venv --python 3.11 && source .venv/bin/activate
uv pip install rasa-pro
rasa init --template tutorial --init-dir .
```

Accetta l'offerta di training. Nota una cosa curiosa rispetto al Day 6: questo training non effettua **alcuna chiamata LLM** — vedremo tra poco il perché, quando leggeremo `config.yml`.

Esamina ciò che hai ottenuto — sta tutto in una schermata:

```
├─ actions/actions.py      # one tiny example custom action
├─ config.yml              # the processing pipeline
├─ credentials.yml         # channels (how users reach the bot)
├─ data/flows.yml          # ONE example flow: transfer_money
├─ data/patterns.yml       # two overridden conversation-repair patterns
├─ domain.yml              # slots + responses, one flat file
└─ endpoints.yml           # where things connect (LLM provider, action code)
```

Confrontalo con il template default del Day 6: nessuna directory `domain/` suddivisa, nessuna `e2e_tests/`, nessun database mock — un solo flow, un unico file di domain piatto, una sola action.

2. Rendiamolo nostro: rimuovere il giocattolo, ripuntare il modello

2.1 Tre rimozioni

Il template fornisce un flow giocattolo `transfer_money`, l'action Python che usa, e due repair pattern sovrascritti. Nessuno di questi è nostro; se ne vanno tutti e tre:

```
rm actions/actions.py data/patterns.yml
```

Poi svuota `data/flows.yml` (lo riempiamo subito dopo) ed elimina da `domain.yml` tutto tranne la riga `version: "3.1"` — gli slot e le responses giocattolo servivano il flow eliminato.

Due di queste rimozioni meritano una parola:

- `actions/actions.py` era l'unica custom action del template — codice Python. Lasciamo deliberatamente la cartella `actions/` al suo posto ma **vuota** (solo `__init__.py`): le custom action sono l'intero argomento del Day 8, e l'obiettivo di oggi è vedere fin dove arriva l'assistente senza di esse.
- `data/patterns.yml` conteneva le sovrascritture del template di due repair pattern integrati (`pattern_chitchat`, `pattern_search`). Una volta rimosso il file, si applicano i default integrati: i messaggi fuori tema ricevono un cortese "can't help with that". Per un assistente bancario che resta sul compito, quel default è un ragionevole punto di partenza.

2.2 L'indirezione del modello: `config.yml` → `endpoints.yml`

Apri `config.yml`:

```
recipe: default.v1
language: en
assistant_id: humid-boundary      # yours will differ – a random id per scaffold
pipeline:
- name: CompactLLMCommandGenerator
  llm:
    model_group: rasa_command_generation_model
  flow_retrieval:
    active: false
policies:
- name: FlowPolicy
```

Leggi l'indirezione: il componente della pipeline (`CompactLLMCommandGenerator`, l'LLM che trasforma i messaggi dell'utente in command) non nomina un modello. Nomina un **model group** — e il gruppo viene risolto in `endpoints.yml`. Guarda lì: il template punta quel gruppo verso un piccolo **modello tutorial ospitato da Rasa**, gratuito da provare, non ciò che vogliamo per il lavoro reale.

Nota anche `flow_retrieval: active: false`. Il flow retrieval è un indice basato su embeddings che seleziona i flow rilevanti da inserire nel prompt dell'LLM quando un progetto ne ha molti; il template lo fornisce disattivato, e alla nostra scala (tre flow oggi) disattivato va bene. È anche la risposta all'enigma sul training di poco fa: costruire quell'indice è l'unica chiamata all'API dell'LLM del training, quindi con il retrieval disattivato `rasa train` gira interamente offline. Le conversazioni hanno comunque bisogno della chiave: ogni messaggio dell'utente passa attraverso il command generator.

Ripunta l'assistente verso OpenAI. In `endpoints.yml`, sostituisci la sezione `model_groups` (ed elimina il blocco `nlg:` sopra di essa, che apparteneva al setup ospitato da Rasa del template):

```
model_groups:
  - id: openai_llm
    models:
      - provider: openai
        model: gpt-5.1-2025-11-13
        reasoning_effort: "none"
        timeout: 15
```

E in `config.yml`, punta il componente verso il nuovo gruppo:

```
llm:
  model_group: openai_llm
```

Nota ciò che **non** c'è qui: la chiave API. Le credenziali del provider vengono lette dall'ambiente (`OPENAI_API_KEY`) — non vivono mai in un file di progetto. Questa separazione — *quale* componente in `config.yml`, dove si connette in `endpoints.yml`, i segreti nell'ambiente — è il pattern per tutto ciò che segue nel corso.

3. Flow 1 — `check_balance`: la macchinaria automatica

La prima capacità: un cliente chiede il proprio saldo. Uno slot, un flow, due responses — e gran parte del comportamento che stiamo per osservare non l'avremo scritto noi.

3.1 Il domain: ciò che l'assistente sa e dice

In `domain.yml`, aggiungi:

```

slots:
  account_type:
    type: categorical
    values:
      - checking
      - savings
    mappings:
      - type: from_llm
  current_balance:
    type: float
    mappings:
      - type: controlled

responses:
  utter_ask_account_type:
    - text: "Which account would you like to check?"
      buttons:
        - title: "Checking"
          payload: "/SetSlots(account_type=checking)"
        - title: "Savings"
          payload: "/SetSlots(account_type=savings)"

  utter_current_balance:
    - text: "Your {account_type} account balance is €{current_balance}."

```

Il trust boundary è ben visibile nelle due mappings. `account_type` è `from_llm`: l'utente lo nomina in testo libero e l'LLM lo compila. `current_balance` è `controlled`: l'LLM non può scriverlo **mai** — può farlo solo la nostra logica (oggi uno stub YAML, domani codice vero). Un assistente che lascia a un language model l'invenzione dei saldi dei conti non è un assistente bancario.

Altri tre meccanismi, tutti nel blocco `responses`:

- **`utter_ask_account_type` è una convenzione di denominazione, non una registrazione.** Quando un flow ha bisogno dello slot `account_type`, Rasa esegue automaticamente la response chiamata `utter_ask_<nome slot>`. Nulla farà riferimento a questa response da nessun'altra parte — il nome è il cablaggio.
- **I bottoni portano payload `/SetSlots`.** Un click invia quella stringa invece del testo libero, e imposta lo slot *in modo deterministico* — nessun modello la legge, non avviene alcuna interpretazione. Il testo libero passa attraverso l'LLM; i bottoni lo scavalcano.
- **`utter_current_balance` interpola gli slot** con la sintassi `{curly}`. La response è la risposta — non c'è alcun "codice di risposta" da nessuna parte.

3.2 Il flow: ciò che l'assistente può fare

Sostituisci il contenuto di `data/flows.yml`:

```
flows:
  check_balance:
    name: check account balance
    description: Look up the current balance of one of the customer's Banca Lumera
accounts.
    steps:
      - collect: account_type
        description: "the account to check: checking or savings"
      - set_slots:
        - current_balance: 2350.75
      - action: utter_current_balance
```

Leggilo dall'alto in basso. La `description` del flow è il suo trigger — è ciò con cui l'LLM confronta i messaggi dell'utente; non c'è alcuna lista di intent. Lo step `collect` chiede lo slot (tramite l' `utter_ask_account_type` eseguito automaticamente) e attende. Lo step `action` esegue la response — le responses sono target `action` legittimi, e per i flow informativi sono l'unica "action" di cui hai bisogno.

E lo step centrale: `set_slots` scrive `current_balance` direttamente da YAML. **Questa riga è una bugia.** Una banca vera cerca il numero nel core banking. Piantiamo la bugia deliberatamente e alla luce del sole: è il punto esatto in cui il Day 8 sostituisce lo YAML con una vera chiamata di sistema. (Nota che soddisfa la mapping `controlled` — `set_slots` e le custom action sono i due scrittori che uno slot controlled consente.)

3.3 Addestra, conversa, leggi i command

```
rasa data validate
rasa train
rasa inspect
```

Nell'Inspector, percorri tre interazioni e leggi i pannelli di destra man mano che procedi:

1. Digita **"what's my balance?"** — il tracker mostra il command emesso dall'LLM: **start flow `check_balance`**. Poi la macchinaria: lo step `collect` esegue `utter_ask_account_type` (non l'hai mai referenziato — il nome ha fatto il lavoro), e i due bottoni vengono resi.
2. **Digita** (non cliccare) **"the savings one, please"** — l'LLM emette **set slot `account_type` = `savings`**: il tuo testo libero è diventato un command strutturato. Poi `set_slots` compila il saldo, e la response interpolata viene resa: *"Your savings account balance is €2350.75."*
3. Riavvia la conversazione e questa volta **clicca il bottone "Checking"** — stesso esito, percorso diverso: il payload `/SetSlots` ha impostato lo slot senza alcun coinvolgimento dell'LLM. Osserva il pannello degli slot: si compila all'istante, e non compare alcun command `set slot` dal modello.

Conta ciò che hai scritto rispetto a ciò che è stato eseguito: due voci di domain e un flow di tre step, contro la domanda automatica, l'estrazione automatica degli slot, l'avanzamento automatico, e un follow-up di completamento ("Is there anything else I can help you with?") che proviene da un pattern integrato. Quel rapporto è la lezione della giornata.

4. Flow 2 — `book_appointment` : il percorso multi-slot

Seconda capacità: prenotare un appuntamento di persona in filiale. Ora la macchineria si ripete — città, argomento, orario — e compaiono due nuovi meccanismi: un **gate di conferma** e la **ramificazione**.

4.1 Prima suddividi il domain

Con una seconda feature, il file piatto `domain.yml` comincia ad affollarsi. Rasa accetta una **directory** `domain/` con la stessa naturalezza di un singolo file, quindi questo è il momento di adottare il layout che scala: un file di domain per ogni feature.

```
mkdir domain
```

Sposta il materiale del saldo in `domain/check_balance.yml` (tutto ciò che è nel §3.1, più l'intestazione `version: "3.1"`), crea `domain/shared.yml` per le cose trasversali alle feature (riceverà contenuto nel §6), ed elimina il file piatto `domain.yml`. D'ora in poi ogni feature porta con sé il proprio file di domain — il ritmo di salto tra file dello sviluppo con Rasa: *file di domain* ↔ *file di flow*, una volta per capacità.

4.2 Il nuovo file di domain

`domain/book_appointment.yml`:

```

version: "3.1"

slots:
  branch_city:
    type: text
    mappings:
      - type: from_llm
  appointment_topic:
    type: categorical
    values:
      - mortgage
      - investments
      - account services
    mappings:
      - type: from_llm
  preferred_time:
    type: text
    mappings:
      - type: from_llm
  appointment_confirmed:
    type: bool
    mappings:
      - type: from_llm

responses:
  utter_ask_branch_city:
    - text: "In which city would you like the appointment?"

  utter_ask_appointment_topic:
    - text: "What is the appointment about?"
    buttons:
      - title: "Mortgage"
        payload: "/SetSlots(appointment_topic=mortgage)"
      - title: "Investments"
        payload: "/SetSlots(appointment_topic=investments)"
      - title: "Account services"
        payload: "/SetSlots(appointment_topic=account services)"

  utter_ask_preferred_time:
    - text: "When would suit you best? Any day and time, in your own words."

  utter_ask_appointment_confirmed:
    - text: "Let me recap: an appointment at our {branch_city} branch, about {appointment_topic}, {preferred_time}. Shall I book it?"
    buttons:
      - title: "Yes, book it"
        payload: "/SetSlots(appointment_confirmed=true)"
      - title: "No, don't"
        payload: "/SetSlots(appointment_confirmed=false)"

  utter_appointment_booked:
    - text: "Done – your appointment is booked. You'll receive a confirmation shortly."

```

```
utter_appointment_declined:
```

```
- text: "No problem – I've discarded the appointment request."
```

Una mossa deliberata qui: la domanda di conferma è essa stessa un `utter_ask_<slot>` — chiede il bool `appointment_confirmed` — e il suo testo **interpola gli altri tre slot**. Il riepilogo e la domanda sono un'unica response. I suoi bottoni scrivono `true / false` in modo deterministico; le risposte digitate ("yes please", "actually no") passano attraverso l'LLM come qualsiasi altro slot.

4.3 Il flow, con una ramificazione

Aggiungi in coda a `data/flows.yml`:

```
book_appointment:
  name: book a branch appointment
  description: Book an in-person appointment at a Banca Lumera branch.
  steps:
    - collect: branch_city
      description: the city of the branch where the customer wants the appointment
    - collect: appointment_topic
      description: "what the appointment is about: mortgage, investments, or account services"
    - collect: preferred_time
      description: the customer's preferred day and time, in their own words
    - collect: appointment_confirmed
      description: whether the customer confirms the recap of the appointment
  next:
    - if: slots.appointment_confirmed
      then:
        - action: utter_appointment_booked
          next: END
    - else:
        - action: utter_appointment_declined
          next: END
```

Nuovi pezzi:

- **next con if / then / else** — la prima ramificazione del corso. Il predicato (`slots.appointment_confirmed`) è valutato dal dialogue engine, in modo deterministico — non è l'LLM a decidere quale ramo viene eseguito.
- **Nulla permane dopo il flow** — in entrambi i casi il flow termina, e gli slot compilati dai suoi step `collect` vengono automaticamente azzerati. Un appuntamento rifiutato non lascia dietro di sé alcuna città o argomento obsoleti: un secondo tentativo richiede di nuovo tutto. (Se un valore dovesse sopravvivere al flow, si tratta di una decisione di progettazione esplicita — un argomento per la giornata sullo stato su larga scala.)
- Ogni collect riceve una `description`: affina ciò che l'LLM estrae (confronta l'"in their own words" di `preferred_time` — oggi accettiamo di proposito testo in forma libera).

Riaddestra e conversa (`rasa train`, poi `rasa inspect`). Un'esecuzione completa:

utente: I'd like to book an appointment at a branch **bot:** In which city would you like the appointment? **utente:** Napoli **bot:** What is the appointment about? (+ *tre bottoni*) **utente:** it's about a mortgage **bot:** When would suit you best? Any day and time, in your own words. **utente:** next Tuesday morning **bot:** Let me recap: an appointment at our Napoli branch, about mortgage, next Tuesday morning. Shall I book it? (+ *bottoni Si/No*) **utente:** yes please, book it **bot:** Done — your appointment is booked. You'll receive a confirmation shortly.

Eseguiamo una seconda volta e rifiuta al riepilogo: ottieni la response di rifiuto, e — prova a prenotare di nuovo — la domanda sulla città ritorna da capo: gli slot sono stati azzerati quando il flow è terminato.

Verifica di onestà, come per lo stub del saldo: **non è stato prenotato nulla da nessuna parte**. La conferma è solo conversazionale. Anche questo è un buco lasciato di proposito per il Day 8.

4.4 Un comportamento che non hai scritto: la correzione

Mentre una prenotazione è in corso, cambia idea:

utente: I want a branch appointment **bot:** In which city would you like the appointment? **utente:** Napoli **bot:** What is the appointment about? (...) **utente:** actually, make it the Milano branch instead **bot:** Ok, I am updating branch_city to Milano. **bot:** What is the appointment about? (...)

Nessuno ha scritto un handler di correzione. L'LLM ha emesso un `set slot` per uno slot *già compilato*, e un repair pattern integrato ha confermato il cambiamento e ripreso dal punto in cui il flow si trovava — il riepilogo alla fine dirà Milano. Questa è la macchinaria dei pattern che lavora sotto ogni flow che definisci; la ottieni gratis.

5. Flow 3 — `card_fees_info` : molte risposte giuste per una domanda

Terza capacità, questa volta onestamente priva di backend: i costi delle carte sono informazioni di riferimento, quindi **le responses sono davvero l'intera risposta**. Il nuovo meccanismo: un unico nome di response, diverse varianti, scelte in base allo stato della conversazione.

5.1 Variazioni condizionali di response

```
domain/card_fees_info.yml :
```

```

version: "3.1"

slots:
  card_type:
    type: categorical
    values:
      - debit
      - credit
    mappings:
      - type: from_llm

responses:
  utter_ask_card_type:
    - text: "Which card are you interested in?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_card_fees:
    - condition:
        - type: slot
          name: card_type
          value: debit
        text: "The Lumera debit card has no yearly fee – it comes free with every current account."
    - condition:
        - type: slot
          name: card_type
          value: credit
        text: "The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year."
    - text: "Every Lumera card has its own fee schedule, listed in the full fee table."

  utter_card_fee_table_link:
    - text: "Full details for your card: https://banca-lumera.example/cards/{card_type}/fees"

```

`utter_card_fees` è **una sola response con tre variazioni**. Ogni blocco `condition` fa match su un valore di slot; l'engine sceglie la prima variante la cui condizione è soddisfatta, e l'ultima variante, priva di condizione, è il fallback — fornisce sempre uno. Nessun LLM sceglie la formulazione: queste sono frasi esatte, approvate dal brand, selezionate in modo deterministico.

E `utter_card_fee_table_link` porta un **deeplink con uno slot dentro l'URL**: `{card_type}` si interpola nel link stesso. Un flow può terminare esattamente nel tipo di deeplink parametrizzato che restituisce un prodotto di ricerca — con la formulazione e l'URL interamente sotto il tuo controllo.

5.2 Il flow

Aggiungi in coda a `data/flows.yml` :

```
card_fees_info:
  name: card fees information
  description: Explain the yearly fees of Banca Lumera payment cards.
  steps:
    - collect: card_type
      description: "the card the customer asks about: debit or credit"
    - action: utter_card_fees
    - action: utter_card_fee_table_link
```

Riaddestra e prova: **"how much does the credit card cost per year?"** — nota che lo step collect *non chiede*: l'LLM ha già estratto `card_type=credit` dal tuo messaggio iniziale, quindi il flow scivola dritto alla risposta:

bot: The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year. **bot:** Full details for your card: <https://banca-lumera.example/cards/credit/fees>

Uno step `collect` chiede solo quando lo slot è vuoto. Chiedi in modo generico ("tell me about card fees") e chiede davvero, bottoni compresi.

5.3 La variante italiana

Banca Lumera serve clienti italiani; le risposte dell'assistente dovrebbero esistere in italiano. Due pezzi. Primo, dichiara la lingua in `config.yml` (a livello top, accanto a `language`):

```
language: en
additional_languages:
  - it
```

Secondo, ogni response porta con sé le proprie traduzioni inline, sotto una chiave `translation` — il `text` di livello top è la lingua di default. Aggiorna le tre varianti di `utter_card_fees` e la response del link in `domain/card_fees_info.yml`:

```

utter_card_fees:
- condition:
  - type: slot
    name: card_type
    value: debit
  text: "The Lumera debit card has no yearly fee – it comes free with every current account."
  translation:
    it: "La carta di debito Lumera non ha alcun canone annuo: è inclusa in ogni conto corrente."
- condition:
  - type: slot
    name: card_type
    value: credit
  text: "The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year."
  translation:
    it: "La carta di credito Lumera costa 45 € l'anno, azzerati se spendi più di 6.000 € nell'anno."
- text: "Every Lumera card has its own fee schedule, listed in the full fee table."
  translation:
    it: "Ogni carta Lumera ha il proprio piano commissioni, riportato nella tabella completa."

utter_card_fee_table_link:
- text: "Full details for your card: https://banca-lumera.example/cards/{card_type}/fees"
  translation:
    it: "Tutti i dettagli per la tua carta: https://banca-lumera.example/cards/{card_type}/fees"

```

In quale lingua gira una conversazione è determinato da uno **slot language integrato** (un nome riservato — non puoi dichiarare tu stesso uno slot chiamato `language`). Impostarlo è compito della tua applicazione — da una preferenza memorizzata, da un parametro di canale, dal locale del browser — e questo è codice, quindi appartiene propriamente al Day 8. Per un rapido test locale, però, la porta deterministica che già conosciamo funziona anche qui — invia questo messaggio nell'Inspector:

```
/SetSlots(language=it)
```

...poi chiedi della carta di debito:

bot: La carta di debito Lumera non ha alcun canone annuo: è inclusa in ogni conto corrente. **bot:** Tutti i dettagli per la tua carta: https://banca-lumera.example/cards/debit/fees **bot:** Is there anything else I can help you with?

Leggi con attenzione la terza riga: è ancora in inglese. Solo le responses che *hanno* una traduzione italiana vengono servite in italiano — le responses dei pattern integrati non ne hanno alcuna. La traduzione è un esercizio di completezza, e Rasa rende la completezza verificabile:

```
rasa data validate translations
```

avvisa, response per response, di ogni traduzione `it` mancante — le nostre sette responses ancora non tradotte e ognuna di quelle integrate. (Il semplice `rasa data validate` mostra lo stesso come due conteggi aggregati.) Oggi lasciamo il resto non tradotto; ciò che conta è il meccanismo.

6. Configurazione della sessione

Un'ultima manopola mentre siamo nel domain: quanto a lungo vive una "sessione" di conversazione. In `domain/shared.yml`:

```
version: "3.1"

session_config:
  session_expiration_time: 60 # minutes of inactivity before a new session starts
  carry_over_slots_to_new_session: true
```

Questi sono i default, scritti per esteso così da renderli visibili: dopo 60 minuti di silenzio un utente che ritorna avvia una nuova sessione, e i valori degli slot vi vengono trasferiti. Se gli slot trasferiti siano giusti per ogni valore (dovrebbe un vecchio `appointment_confirmed` sopravvivere?) è una questione di progettazione che incontreremo di nuovo quando la gestione dello stato avrà la sua giornata dedicata.

7. Dove siamo arrivati

Layout finale — confrontalo con lo scaffold di una schermata da cui siamo partiti:

```
|— actions/                # EMPTY on purpose – Day 8's subject
|— config.yml              # + additional_languages, openai_llm group ref
|— credentials.yml
|— data/flows.yml          # three flows, all YAML
|— domain/
|   |— shared.yml          # session config
|   |— check_balance.yml
|   |— book_appointment.yml
|   |— card_fees_info.yml
|— endpoints.yml           # openai_llm model group; key stays in the env
```

Tre capacità, zero Python. Ciò che è stato eseguito senza che l'avessimo mai scritto: la domanda automatica (`utter_ask_<slot>`), l'estrazione automatica (`from_llm` + `command SetSlot`), l'input deterministico dai bottoni (`/SetSlots`), l'avanzamento automatico dei flow, la gestione delle correzioni, i follow-up di completamento, il fallback per i fuori tema, la selezione delle response in base allo stato, il serving delle traduzioni.

E due bugie oneste, piantate alla luce del sole, entrambe in attesa del Day 8:

1. `set_slots: current_balance: 2350.75` — il saldo è inventato in YAML; nessun sistema bancario è stato consultato.
2. "Done — your appointment is booked" — non è stato prenotato nulla da nessuna parte.

Il Day 8 sostituisce entrambe con vere chiamate a un'API di core-banking (mock) — ed è lì che Python entra finalmente nel progetto.

8. Extra — attivare il chitchat (solo demo, non conservato)

Extra. Questa sezione è una demo dal vivo, non fa parte del progetto. Alla fine annulliamo entrambe le modifiche; lo snapshot del Day 8 parte senza di esse.

Due cose che abbiamo eliminato prima ricevono ora la loro spiegazione: le sovrascritture di `data/patterns.yml` del §2.1, e il blocco `nlg:` del §2.2. Prova prima l'assistente: di' qualcosa fuori tema — "do you like pizza?" — e il `pattern_chitchat` integrato devia con un cortese "can't help with that". Quella deviazione è un flow fornito da Rasa; come qualsiasi pattern, possiamo sovrascriverlo definendo un flow con lo stesso nome. Ricrea `data/patterns.yml`:

```
flows:
  pattern_chitchat:
    description: handle interactions with the user that are not task-oriented
    name: pattern chitchat
    steps:
      - action: utter_free_chitchat_response
```

I flow sono artefatti addestrati, quindi questo richiede un `rasa train`. Chiedi di nuovo della pizza — e la risposta è una *stringa fissa*: "Sorry, I'm not able to answer that right now." Strano, per una response chiamata "free chitchat". Il motivo: l' `utter_free_chitchat_response` integrato è marcato `rephrase: True` nei suoi metadata, ma la riformulazione è svolta da un componente che non abbiamo — il **contextual response rephraser**, un LLM che riscrive le response templated nel contesto. Vive nel livello di response, che non è né la `pipeline:` né le `policies:` — è configurato in `endpoints.yml`, esattamente nello slot `nlg:` che abbiamo eliminato nel §2.2:

```
nlg:
  type: rephrase
```

Nessun riaddestramento — gli endpoints sono cablaggio di runtime, quindi riavvia il server e chiedi ancora una volta della pizza. Ora l'assistente chiacchiera davvero in risposta, nel contesto. Leggi ciò che ha appena attraversato l'intera architettura: la **pipeline** (command generator) ha classificato il messaggio come `Chitchat`, le **policies** (FlowPolicy) hanno eseguito `pattern_chitchat`, e il **livello NLG** ha generato le parole. Tre fasi, tre case di configurazione.

Perché non lo conserviamo: la generazione in forma libera significa che l'LLM potrebbe tranquillamente rispondere ben al di fuori dell'ambito bancario — la documentazione avverte esattamente di questo. Per un assistente che dovrebbe restare sul compito, il default che devia è la

scelta giusta, quindi elimina di nuovo `data/patterns.yml` e il blocco `nlg:`, e riaddestra. Incontreremo `pattern_chitchat` un'altra volta, più avanti nel corso, quando romperemo l'assistente di proposito e congeleremo le sue riparazioni in dei test.